Distill II

Version prompts and test grounded answers

Prompts, generation and evaluation gates Session 3 of 5 · 3 hours · source-aligned workshop

What you will produce

- A grounded answer replay and a failure diagnosis.
- Three proposed golden cases with a review memo.
- A documented quality gate decision.

The three-hour plan

25 min prompts/evaluation + 20 min code walkthrough

90 min pair labs, including a 10 min break

25 min gate decisions + 20 min review

Showcase / reject a convincing bad answer

PROMPT INPUT

An attacker supplies closing tags and instructions.

GENERATED DRAFT

A generated draft cites an invented quote.

EVALUATION CANDIDATE

A candidate has a lower faithfulness score.

Run prompt and evaluation demonstrations

```
python -m training.showcase --session 3
```

Tag escape retained; bad quote → abstention.

Same cases: −0.25; changed dataset: refused.

Prompt Engineering and Prompt Life Cycle Management

Educative lesson 09

Concept → worked example → NexaAssist application

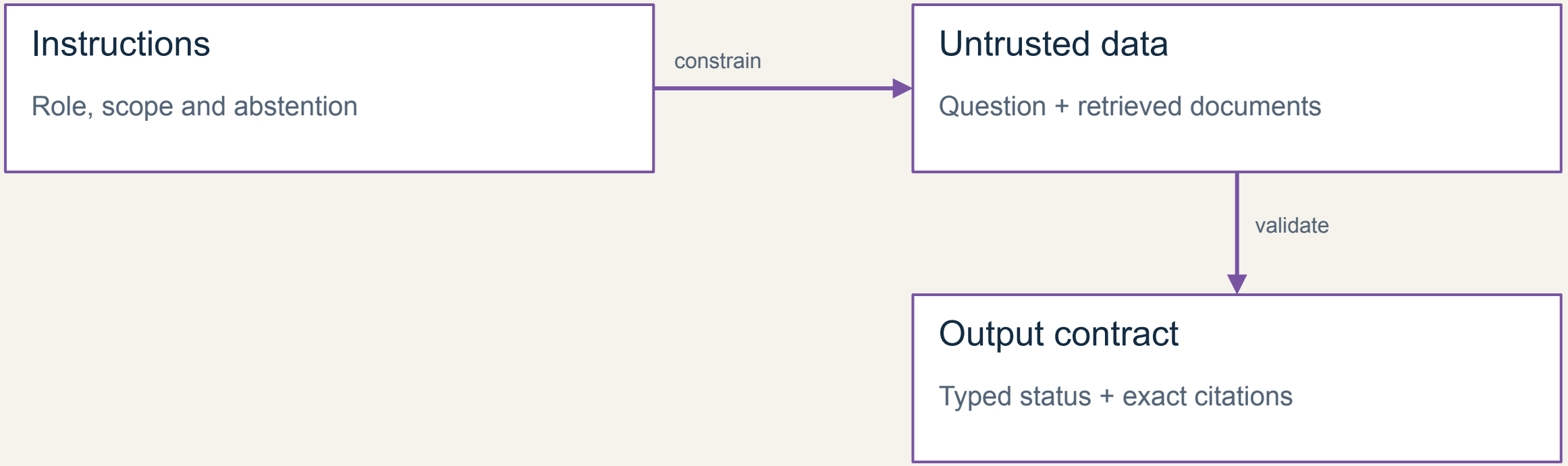
A prompt has three responsibilities

> Role: what the assistant is authorized to do.

> Evidence boundary: which supplied text supports claims.

> Output contract: schema, status and citations.

A production prompt separates three concerns



Delimiters mark structure; they do not stop every attack

> Wrap untrusted content with explicit boundaries.

> Escape delimiter-like content before rendering examples.

> Keep privileged instructions outside the data field.

The sandwich pattern repeats a narrow rule

- Give scope and grounding instructions before the data.
- Place the question inside its declared data boundary.
- End with a reminder to follow the output contract.

JSON mode and strict schema solve different problems

> Valid JSON only guarantees a parseable representation.

> A schema checks fields, types and allowed status values.

> Source support and policy meaning need further checks.

The prompt is a tracked artifact

```
ANSWER_PROMPT_ID = "nexaassist-policy-answer"
ANSWER_PROMPT_VERSION = "1.1.0"
ANSWER_PROMPT = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            """
You are NexaAssist, an assistant that explains synthetic NexaBank policies to staff.

Resolve product/date/rate ambiguity before judging whether evidence is sufficient.
If the user's requested policy-lookup date differs from as_of, ask for scope
```

Prompt ID, version and fingerprint travel with the answer provenance.

Make a prompt change reviewable

> Run tests/test_prompts.py before changing wording.

> A wording change requires a version and pin update.

> Then replay cases; a new hash proves identity, not quality.

LCEL composes prompt and strict model output

```
generation = ANSWER_PROMPT | llm.with_structured_output(
    AnswerDraft, method="json_schema", strict=True
)
```

The chain requests AnswerDraft with strict JSON-schema output.

Status is part of the API contract

```
class AnswerDraft(BaseModel):
    """Model output; source identities are resolved by application code."""

    model_config = ConfigDict(extra="forbid")

    status: Literal["answered", "insufficient_evidence", "needs_clarification"]
    answer: str = Field(min_length=1)
    evidence: list[EvidenceQuote]
```

Distinguish answered, insufficient_evidence and needs_clarification.

A registry supports comparison and rollback

> Track prompt text, version, owner and evaluation evidence.

> Compare candidates with the same cases and configuration.

> Hot-swapping and A/B routing need their own controls.

Practical / render a boundary attack safely

```
Run: python -m training.session3.prompt_boundaries
```

Inspect escaped closing tags and retained question text.

Check template variables and compare the production rendering.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

The answer path fails closed

1. Retrieve and budget evidence → generate draft
2. Check output PII → check IDs, markers and exact quotes
3. Return supported response, repair once or abstain

What exact quotes do and do not establish

> An exact quote proves those bytes were supplied.

> It does not prove the claim follows from the quote.

> Semantic evaluation must still check grounding and scope.

Practical 1 / replay three model drafts

```
Run: python -m training.session3.answers
```

Valid → answered; invented quote → abstention.

Output PII → abstention with no leaked citation.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Exercise / break evidence identity

```
In a lab copy, change the evidence ID to 99.
```

Predict the marker/ID mismatch and repair count.

Then explain a wrong answer with a valid quote.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Evaluation of LLM Outputs: Metrics, Tests, and Human Feedback

Educative lesson 10

Concept → worked example → NexaAssist application

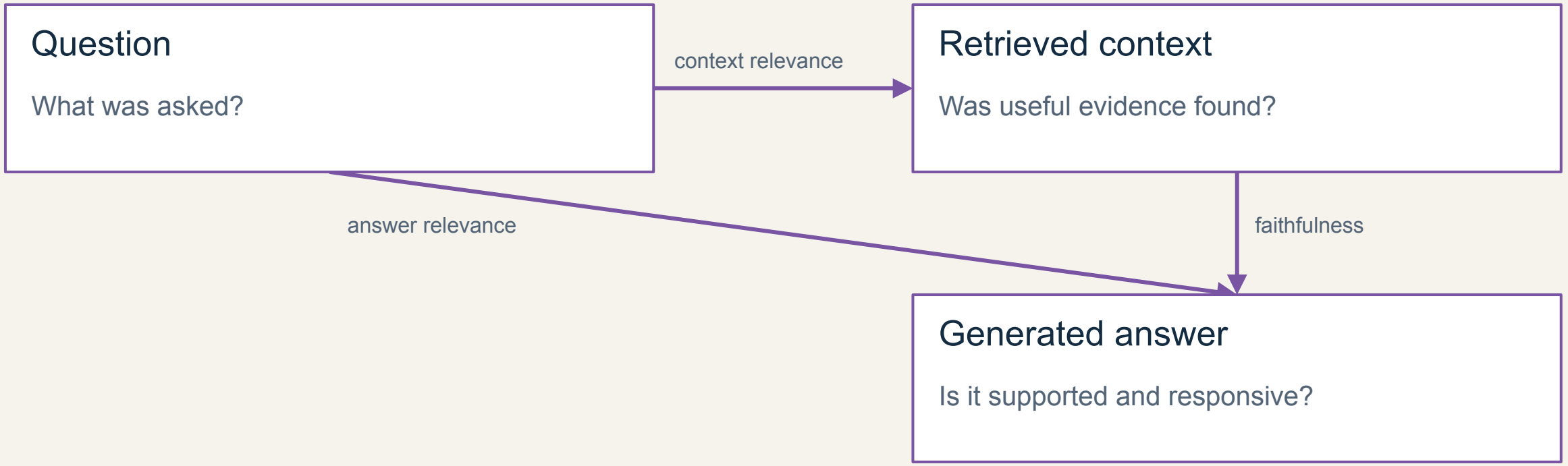
Meaning defeats exact string comparison

> Paraphrases can be correct despite little word overlap.

> A tiny negation can reverse an otherwise similar answer.

> Use semantic criteria plus deterministic contract checks.

The RAG triad assigns different questions



Use metrics that match the failure

- Context precision: relevance of retrieved material.
- Context recall: coverage of reference information.
- Faithfulness and answer relevance: grounding and fit.

Golden cases need human verification

> Input includes question, date and product scope.

> Expected output/status and source references are explicit.

> Put verifier and review rationale in a companion memo.

A judge needs a rubric and calibration

- Provide query, context, response and explicit criteria.
- Score claims against evidence and record reasons.
- Compare a sample against human judgments.

Practical 2 / classify the fault

```
Run: python -m training.session3.gates
```

Missing required context → retrieval_error.

Complete context, unsupported response → generation_error.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

A gate names the metric that failed

```
def gate_failures(report: dict[str, Any], gates: Gates) -> list[dict[str, Any]]:
    """Offline quality gate: aggregate means plus the cases that fell short."""
    records = report["records"]
    failures: list[dict[str, Any]] = []

    def check(metric: str, value: float, minimum: float) -> None:
        if value < minimum:
            failures.append({"metric": metric, "value": round(value, 4), "minimum": minimum})

    for name, value in report["ragas"]["means"].items():
        check(name, value, getattr(gates, f"{name}_min"))
    for key in ("correctness", "groundedness"):
```

Per-metric means and failed cases make the decision actionable.

Practical 3 / propose three golden cases

```
Copy the JSONL dataset into /tmp.
```

Add answer, abstention and clarification examples.

Validate with load_golden; review the actual source clauses.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Offline does not always mean no provider

> Class labs use fakes and need no credentials.

> A pre-release golden evaluation can call real models.

> Live evaluation needs approved access, cost and data.

Compare like with like

> Keep case set, retrieval configuration and judge aligned.

> Save the baseline before writing the next report.

> Inspect per-metric deltas and newly failing cases.

Spend evaluation budget where it catches regressions

A representative subset can gate frequent changes.

Broader release or scheduled evaluations cover more cases.

Keep judge, dataset and prompt identities in reports.

Optional live / run and compare

> Run app.evaluation.run --limit 3 with approved .env.

> Preserve old and new reports under distinct paths.

> Run app.evaluation.compare OLD.json NEW.json.

Exit gate / quality with an explanation

- Three valid cases and human review memo.
- A named failure type for every failing example.
- Thresholds met or a documented release hold.

Next / make failure safe to serve

> Keep generation behind the service layer.

> Bring one invalid input and one dependency outage.

> Session 4 will measure the API failure contract.